

Command Center Role & Routing Spec

Every role: what puts a patient in the queue, what the rep must complete before the stage can advance, and exactly where each outcome sends them. Read from the code, not from prior docs.

Version 1.0 Issued 11 Aug 2026 Source commit b5a5f8f

00 How to read this

Each role is one spec block with the same six fields. **Entry** is the predicate that puts a patient in the queue. **Due** is the snooze rule that decides whether they surface today. **Gates** are what must be filled before the primary action unlocks. **Exits** is the routing table — every outcome, the columns it writes, and the destination.

Three rules apply to every role

1 · THE ADVANCER IS WRITTEN LAST

Every "Send to Monday" runs `executeWritesWithVerification`: snapshot → write data columns → poll read-back (8 tries, ~12s) → **only then** write the stage advancer. If verification times out it throws and **the advancer is never written**. The patient stays put by design, because the automation would otherwise read stale siblings.

2 · ESCALATION REMOVES A PATIENT FROM THE REP

An escalated patient leaves the role's list *and* its count and appears in a manager column. **DVS is the only exception** — it ignores escalation entirely. On the Medical Evaluation board, escalation index `2` (proposed stuck) is excluded from *both* the active and the escalated counts.

3 · MOVEMENT BETWEEN BOARDS IS A MONDAY AUTOMATION

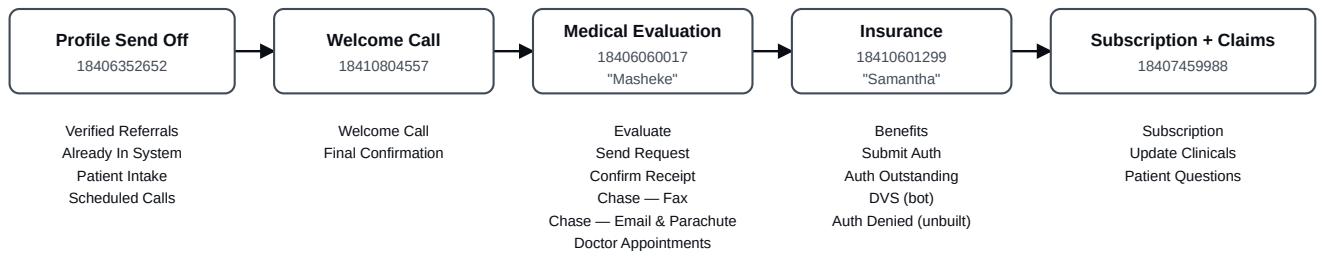
The app only ever flips a status column. Board-to-board movement, item creation on the next board, and group moves are board automations. On the **Medical Evaluation board this is not true of sub-stages**: `Sub-Stage color_mm1wyr92` is the advancer, so moving between Evaluate → Send Request → Confirm Receipt → Chase → Doctor Appointment needs no automation at all.

Destination legend

ADVANCES moves to the next stage · **STAYS** remains in this queue, usually snoozed · **ESCALATES** leaves for a manager column · **TO A BOT** handed to an automated stage

01 The pipeline

Five boards, nineteen roles. Everything below is a detail of this picture.



STAGE ADVANCER PER BOARD

Profile: Move to Onboarding · Welcome Call: Stage Advancer · Medical Evaluation: Sub-Stage color_mm1wyr92 (no automation needed)

Insurance: Stage Advancer color_mm1ws96t — index 0 Auth Denied · 1 DVS · 2 Stuck · 3 Benefits/SoS · 4 Submit Auth · 6 Auth Outstanding · 7 Complete

02 Profile Send Off

Board 18406352652

groups: `1. Intake` `group_mm1xf2jb` · DTC form `group_mm5zgeak` `group_mm5z87zt` · `Patient Intake`
`group_mm4vhqff`

CORRECTED 10 AUG 2026 — CLAUDE.MD §5.10 IS STALE ON THIS

The `1. Intake` group is **no longer split by referral type or source**. Patient Intake is now the DTC form's own two groups and nothing else. `referralsplit.ts` retains `isUnverified` but the counts hook explicitly discards it (`void isUnverified`) — it survives only because Oversight still labels by it.

Verified Referrals

id `profile` · `/profile`

ENTRY Group `1. Intake` , and *Already In System* `color_mm2xe7r8` $\neq$ `Yes` .

DUE *Follow Up* `color_mm3822qq` $\neq$ `Done` .

GATES **Run Stedi Check is gated on the profile writing first.**
`writePatientProfile` then `verifyProfileWritten` (≤ 3 tries) must succeed — Stedi reads Name, DOB, General Insurance and Member ID
`text_mm4t8gbq` *from Monday*, so if verification fails the run aborts and **Stedi never fires**.

Results are polled every 4s and revealed only after the full result-column set has been stable for 10s. Hard timeout 95s.

EXITS	OUTCOME	WRITES	DESTINATION
	Advance to MN	Profile data columns, then <i>Move to Onboarding</i> = <code>Advance to MN</code> (written last)	ADVANCES automation creates the Medical Evaluation item and moves this one to Completed
	Send back to Patient Intake	<code>moveItemToGroup(group_mm4vhqff)</code> — not a Move to Onboarding write	STAYS on this board, different group

NOTES The Stedi UI is inline in `ProfilePage.tsx` .
`components/profile/StediPanel.tsx` is dead code with zero importers — editing it changes nothing. Adding a Stedi result column touches 5 places, all listed in CLAUDE.md §5.11.

`lib/profile/mondayWrite.ts` · `lib/profile/referralSplit.ts` · `pages/ProfilePage.tsx`

Already In System

id `inSystemReferrals` · `/in-system-referrals`

ENTRY Group **1. Intake**, *Already In System* = **Yes**. Checked **before** everything else, so it wins over any other classification.

DUE *Follow Up* ≠ **Done**.

EXITS Same two as Verified Referrals — the page is the same component with a different **variant**.

NOTES A blank *Already In System* counts as **not** in system. The column is not always set.

Patient Intake — DTC & CareCentrix

id `unverifiedReferrals` · `/unverified-referrals`

ENTRY The **two DTC form groups** (`group_mm5zgeak`, `group_mm5z87zt`). Every item there came from the form, so no referral split is applied — they are unverified by definition.

DUE *Follow Up* ≠ **Done** and not intake-escalated (`color_mm5zww42` ∉ {Manager Escalation Required, Final Escalation Required}).

NOTES The role id stays `unverifiedReferrals` so existing `access.json` assignments keep working. Same reason `profile` kept its id when relabelled.

Patient Intake — Scheduled Calls

id `scheduledCalls` · `/scheduled-calls`

ENTRY The same two DTC form groups, but counted by **clock, not by a follow-up rule**.

DUE Booked date `date_mm63na19` = today **and** booked time $\geq$ now (ET) **and** booking status `color_mm5zrbn3` $\neq$ `Canceled`. Booked today with **no time on file still counts** — it can't be sequenced, so it can never be "passed".

NOTES The count falls through the day as start times pass. The 9 AM baseline lands before any appointment has passed, so it captures the day's full total and this is a real burndown rather than a coincidence.

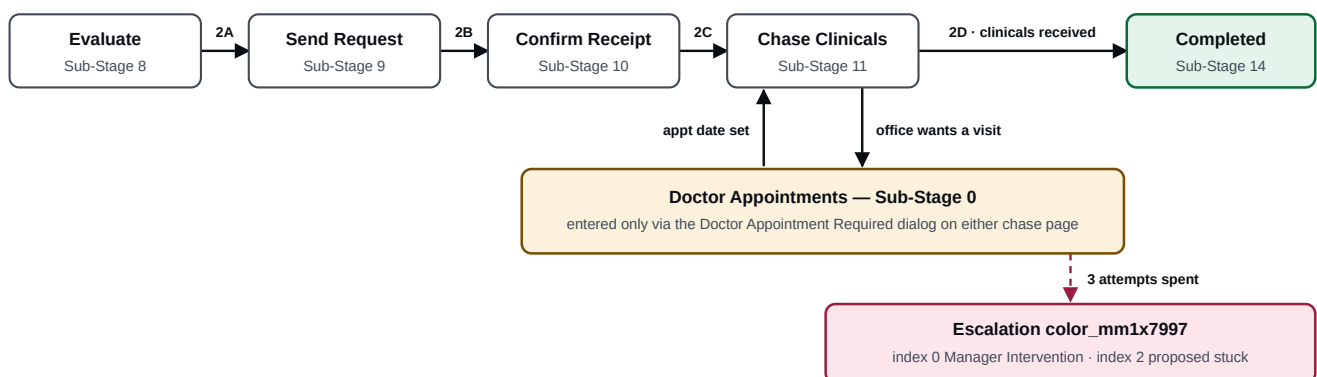
`lib/scheduledCalls/workflow.ts` `remainingToday` · `hooks/useRoleCounts.ts`

03 Medical Evaluation

Board 18406060017 · "Masheke"

one group, five sub-stages. *Sub-Stage* `color_mm1wyr92` IS the advancer

Six roles on one group. Sub-Stage indices are **not sequential**: Evaluate 8, Send Request 9, Confirm Receipt 10, Chase 11, **Doctor Appointment 0**, Completed 14. Index 0 is not a typo — Monday assigned the lowest free slot when the label was created in the UI. A status write by index is **silent** if the index has no label, so verify `settings_str` before changing it.



Chase splits into two roles by Clinicals Method `color_mm1xw7y5` — set back at Send Request. Parachute or Email → Email & Parachute; everything else including blank → Fax.

Evaluate

id `evaluate` · `/evaluate` · Sub-Stage `Evaluate MN`

ENTRY Sub-Stage text exactly `Evaluate MN` .

DUE *Next Action Date* `date_mm1wadgs` blank, today, or past. Escalation index 0 → escalated list; index 2 → counted nowhere.

GATES The Send button is blocked until `deriveValidity` resolves every visible requirement. The value written to *Medical Necessity* comes from the same derivation, so **the stage move can never disagree with the written column**.
Render gate: the CGM/IP Coverage Path and Language controls only appear once that product's script is Received (Yes) or Invalid. Already-saved values still write on send via `buildScriptCoverageWrites` .

EXITS	OUTCOME	WRITES	DESTINATION
	Send to Monday	~16 eval status columns + MN notes, then <i>Advancer 2A</i> → Complete and <i>Sub-Stage</i> → 9	ADVANCES Send Request
	Propose Stuck	Escalation → index 2, reason stamped into MN notes as <code>[Proposed Stuck ...]</code>	ESCALATES Final Decisions

NOTES **"Option A" encoding:** the board has no per-requirement columns. Per-requirement Yes/No/Invalid answers round-trip through two existing dropdowns (*IP MN Invalid Reasons*, *IP MN No Reasons*) by **exact label-string match**. A casing mismatch makes Monday silently create a duplicate label. Edit `IP_REQ_LABELS` only against the live board; two round-trip test suites guard it.

lib/masheke/evalState.ts · mondayWrite.ts `sendPatientToMonday("evaluate")`

Send Request

id `sendRequest` · `/send-request` · Sub-Stage `Send Request`

ENTRY Sub-Stage text exactly `Send Request` .

DUE Next Action Date rule, as above.

EXITS	OUTCOME	WRITES	DESTINATION
	Send to Monday	Generate CGM Script, Generate IP Script, Clinicals Method , MN notes, then <i>Advancer 2B</i> → Complete and <i>Sub-Stage</i> → 10	ADVANCES Confirm Receipt

NOTES

THIS ACTION GATES A LATER ONE

Clinicals Method is set here and decides which Chase queue the patient lands in two stages later. Leaving it blank is not neutral — blank routes to Chase — Fax. That is deliberate (nobody falls through the cracks), but it means a Parachute patient whose method was never set will be worked as a fax patient.

Dispatch is via the Cloudflare worker `POST /send-message` . Recipients may be normal emails or `<number>@rcfax.com` , which RingCentral converts to a fax. All email recipients go out as **one grouped message**; each rcfax recipient gets its own, so the fax addresses are never exposed to human recipients.

Confirm Receipt

id `confirmReceipt` · `/confirm-receipt` · Sub-Stage `Confirm Receipt`

ENTRY Sub-Stage text exactly `Confirm Receipt` .

EXITS	OUTCOME	WRITES	DESTINATION
	Send to Monday	MRs/Clinicals, MN Attempts, Receipt Confirmed Date + Name, notes, then <i>Advancer 2C</i> → Complete and <i>Sub-Stage</i> → 11	ADVANCES Chase Clinicals

NOTES One of two panels using **blocking saves** (`requireDone` + `SaveProgressOverlay`): the screen is locked, sidebar included, until Monday confirms. This exists because of the July 2026 dropped-date incident — a mid-save patient switch clobbered panel state and silently dropped the Next Action Date, so completed patients never left the due queue and burned attempts.

The follow-up date is computed **at save time**, never from component state, and a missing date aborts the save loudly.

Chase Clinicals — Fax / Email & Parachute

ids `chaseFax` `chaseParachute` · `/chase-fax` `/chase-parachute`

ENTRY Sub-Stage `Chase Clinicals`, then split on *Clinicals Method*
`color_mm1xw7y5`: `Parachute` or `Email` → `chaseParachute`; everything else including blank → `chaseFax`.

DUE Next Action Date blank/today/past.

GATES The attempt counter is *MN Attempts* `color_mm1wz0vg`, advancing Attempt 1 → 2 → 3 → `Escalate`. Once it reads `Escalate` the panel is **locked to reps** and only a manager can act.

EXITS	OUTCOME	WRITES	DESTINATION
	Log attempt 1 or 2	MN Attempts → next slot, Next Action Date +3 business days , note appended	STAYS snoozed 3 business days
	Log attempt 3	MN Attempts → <code>Escalate</code> , Escalation → index 0	ESCALATES Manager Intervention
	Clinicals received	MRs/Clinicals → MR Received, <i>Advancer 2D</i> → Complete	ADVANCES Completed → Insurance board
	Doctor Appointment Required, no date	Sub-Stage → 0, Escalation → Done	STAYS Doctor Appointments queue
	Doctor Appointment Required, date known	Appointment Date <code>date_mm5w2vsf</code> , NAD = appt+1 (weekend-clamped), Escalation → Done	STAYS in Chase, snoozed to the visit

NOTES **+3 business days applies to every Clinicals Method** — Fax, Email, Parachute and blank alike (`nadBumpDays = 3`).

Email deliberately queues with Parachute but is still **sent by email**; the optional re-send box keys off the panel's `roleMethod`, so Email patients never see the fax re-send path. This grouping is applied in **five** places that must agree — role page, `useRoleCounts`, Oversight `CHART_FILTERS`, and both baseline generators.

Also a blocking save (`requireDone` + overlay), same incident as Confirm Receipt.

Doctor Appointments

id `doctorAppointments` · `/doctor-appointments` · Sub-Stage `Doctor Appointment` (index 0)

ENTRY

Only via the Doctor Appointment Required dialog on a chase page. There is deliberately no way for a rep to set this on their own judgment. One rule is the entire state machine: **Appointment Date set ⇒ a normal Chase patient snoozed to appt+1. Blank ⇒ this queue.**

DUE

Next Action Date rule. A patient whose Appointment Date is today-or-later is in *Scheduled* and nowhere else, whatever their NAD says.

GATES

A note is mandatory on every attempt (`canLogAttempt`). The attempt count is derived by parsing lines out of MN Workflow Notes — there are no per-attempt columns — so a note-less save would be indistinguishable from no attempt and hand the rep unlimited retries.

The 3-attempt cap is a **rep guardrail only** (`apptCapApplies`). An escalated patient has no limit; the manager working them logs as many as it takes.

EXITS

Three ways out, and only three.

OUTCOME	WRITES	DESTINATION
Patient booked an appointment	Appointment Date, Sub-Stage → 11, NAD = appt+1 (floored at today), Escalation → Done	ADVANCES back to Chase
Third attempt spent	Escalation → index 0 , never index 2	ESCALATES Manager Intervention
Won't schedule / wants to cancel	Propose Stuck at any attempt — climbs the shared ladder, reason stamped with stage and attempt number	ESCALATES one rung up
No answer · left message	Attempt line appended, NAD +1 business day	STAYS snoozed 1 business day
Spoke — patient will call the office	Attempt line appended, NAD +7 calendar days	STAYS snoozed one week

The attempt line is the counter. Format is exactly

8/3/26, 1:38 PM · Phone call – No answer / no response · <note> –JH .
 apptAttemptsFromNotes matches {known method} – {known outcome} and numbers by position. A format change not mirrored in the parser does not error — it **silently resets the count**.

The count is read only from lines **after the last reset marker** (stage-entry stamp, or [Returned to queue]). Without that, a patient re-entering the stage or handed back by a manager would arrive already capped.

Why a refusal doesn't wait for the third attempt: it's a rep judgment about what they were told, not a counter running out. It climbs one rung rather than jumping to Final so a manager reviews the refusal before the patient can leave the pipeline.

Entry clears escalation because the column is shared. Without it, a manager working an escalated chase patient who clicks the button delivers them into this queue already escalated — and escalated patients are hidden from this sidebar, so they'd be invisible on arrival with no error.

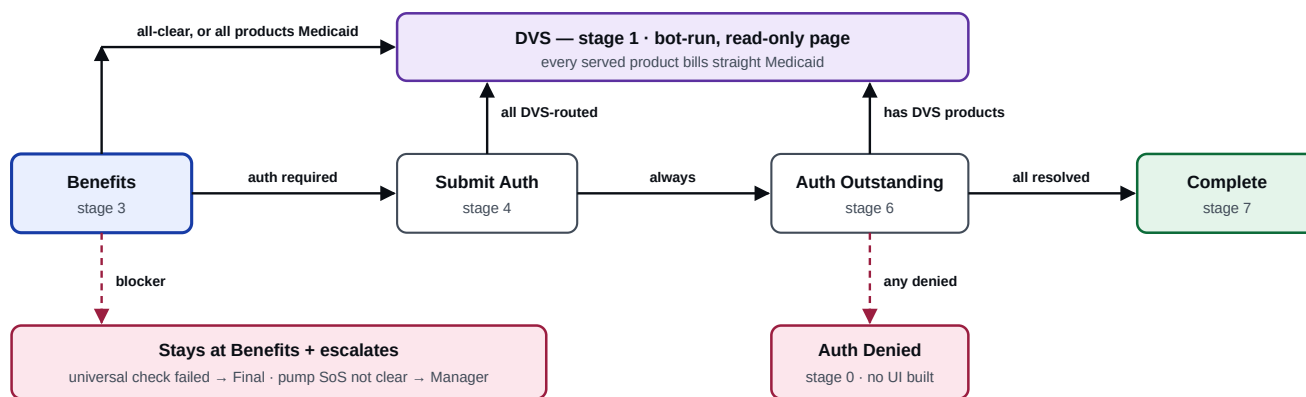
Sidebar sections differ by **view, not access level**: *Awaiting reply* and *Scheduled* render only when `?mv=` is present (set by every Oversight column and nothing else). The manager view lists **every** patient in the stage including escalated ones.

```
lib/masheke/apptOutreach.ts (+ tests) · mondayWrite.ts enterDoctorAppointments ,
returnToChaseWithAppointment , scheduleAppointmentFromChase
```

Board 18410601299 · "Samantha"

Stage Advancer `color_mm1ws96t` · Escalation `color_mm2vsh2f` · Follow Up `color_mm34jz1x` / date `date_mm34m2dz`

Unlike Medical Evaluation, the Stage Advancer here **does** drive board automations, and the routing is computed rather than linear. Stage-DVS items are dropped from the Benefits / Submit Auth / Auth Outstanding counts even though they linger in those groups — there is no group-move automation.



Benefits writes Escalation unconditionally (including Done) because it is DERIVED from the universal checks. Submit Auth never writes it. Auth Outstanding writes it only when an auto rule fires.

ENTRY Benefits group, and Stage Advancer text ≠ `DVS` .

DUE Snoozed only when *Follow Up* = `Follow up` **AND** the Follow Up Date is blank or in the future. **A "Follow Up" whose date has arrived is due again.**

GATES `validateBenefitsForSubmit` returns a list of missing labels; a non-empty list blocks the send:

- All **three universal checks** picked: In-Network Confirmed, Insurance Active, DME Benefits Confirmed.
- Each **visible** served product must have **both** Auth Requirements and Same or Similar. Medicaid-routed supplies are hidden and auto-filled, so they're skipped.

EXITS Computed by `deriveInsuranceOutcome` , evaluated in this order:

OUTCOME	CONDITION	DESTINATION
blocker	Any universal check explicitly not confirmed, or pump SoS = not clear	<code>ESCALATES</code> stays at stage 3; universal fail → Final, pump-SoS-only → Manager
incomplete	Checks pending, or any served product missing Auth or SoS	<code>STAYS</code> stage 3
auth-required	Any product Auth = required	<code>ADVANCES</code> Submit Auth, stage 4
all-clear	Everything confirmed, no auth needed	<code>ADVANCES</code> Complete, stage 7
all products DVS-routed	Overrides the two above when outcome is neither blocker nor incomplete	<code>TO A BOT</code> DVS, stage 1

NOTES

Benefits is the one stage that writes Escalation unconditionally, "Done" included. Escalation here is derived from the universal checks, so clearing it by fixing the facts and re-sending is the design. Every other Insurance stage leaves the column alone unless an auto rule fires.

Landing at DVS also auto-flips a bot trigger by serving: pump first for straight-Medicaid pump patients, supplies otherwise.

BenefitsPatientHeader is **read-only for everyone, managers included.** A manager edit dialog existed from 2026-07-30 and was removed — changing the payer means re-verifying eligibility, and the Insurance board cannot run a Stedi check (no trigger column, neither input column, only ~9 of 33 result columns). Corrections go back through Profile Send Off.

lib/samantha/workflow.ts `deriveInsuranceOutcome`, `validateBenefitsForSubmit` · mondayWrite.ts

Submit Auth

id `submitAuth` · `/submit-auth` · stage index 4

ENTRY Submit Auth group, Stage ≠ `DVS`.

DUE Same status-plus-date rule as Benefits.

GATES `validateSubmitAuthForSubmit`, per submission card:

- Submission Method must be picked.
- If the method is `Call` or `Fax`, a number is required.
- Submission Date is required.

EXITS	OUTCOME	CONDITION	DESTINATION
	Auth Submission Complete	Any product not DVS-routed	<code>ADVANCES</code> Auth Outstanding, stage 6
	Auth Submission Complete	<code>allProductsDvsRouted</code> — zero submission cards because everything bills straight Medicaid	<code>TO A BOT</code> DVS, stage 1
	Propose Stuck	Rep action, any time	<code>ESCALATES</code> Manager Intervention

NOTES

The send never touches Escalation. Submit Auth has no auto rule and no toggle, which is exactly what keeps a rep's Propose Stuck sitting at Manager until a manager actually reviews it.

The send sets **Follow Up Date = today (ET)**, not +1 — many auths approve immediately. This is the prerequisite for the Auth Outstanding date bucket. The Follow Up *status* column is deliberately left alone.

lib/samantha/submitAuthRules.ts · mondayWrite.ts

Auth Outstanding

id `authOutstanding` · `/auth-outstanding` · stage index 6

ENTRY Auth Outstanding group, Stage ≠ `DVS`.

DUE **Pure date bucket.** Snoozed only when Follow Up Date is in the future. The Follow Up *status* column is ignored entirely and a **blank date counts as due**.

GATES Cards appear for products whose board auth status is exactly `Submitted`, or `No Auth Needed` while still SoS-deferred. DVS-routed products never get a card.

EXITS `authOutstandingOutcome` — a strict priority ladder, first match wins:

CONDITION	STAGE WRITE	DESTINATION
1 · Any product denied	stage 0	<code>ESCALATES</code> Auth Denied + Escalation → Manager
2 · Pump SoS not clear	<code>none</code> — stage left alone	<code>ESCALATES</code> Final Decisions
3 · All resolved, has DVS-routed products	stage 1	<code>TO A BOT</code> DVS
3 · All resolved, no DVS products	stage 7	<code>ADVANCES</code> Complete
4 · All remaining are DVS-routed	stage 1	<code>TO A BOT</code> DVS
5 · Otherwise	<code>none</code>	<code>STAYS</code> re-dated, no escalation

NOTES A denial writes **Manager**, not Final — the patient leaves for the unbuilt Auth Denied stage, so choosing Final would pre-judge a design nobody has done. Auth Outstanding is the one Insurance stage with **only a Final Decisions manager chart**, no Manager Intervention chart. Because that single chart must catch everything, its population rule accepts **either** escalation index. A nightly job recalculates *Days Auth Outstanding* `numeric_mm5f5ars` as days since the earliest per-product Auth Submission Date. Idempotent recalc, not an increment.

lib/samantha/authOutstandingReview.ts `authOutstandingOutcome` · `authOutstandingDays.ts`

DVS

id `dvs` · `/dvs` · stage index 1

ENTRY

Stage Advancer index 1 **board-wide** — there is no dedicated group, so items are found by the status value alone.

DUE

Date-only, same as Auth Outstanding.

EXITS

None from the app. **The page is a read-only monitor; the bot does the work.** Four board automations escalate on failure — Trigger Supplies DVS, Trigger Pump DVS, S Claims Status, IP Claims Status — each writing Escalation = Manager Escalation Required.

NOTES

THE ONE EXCEPTION TO THE ESCALATION RULE

Escalated patients stay in the DVS queue and the DVS count. The queue keys purely off the DVS and claims status columns, and no automation flips DVS patients to a manager escalation, so a label carried in from an earlier stage must not hide them. The escalated count is reported as `0` deliberately — the "all" filter sums active + escalated and relies on them being disjoint.

Supplies and pump claims are **separate columns** (`S Claims Status` / `IP Claims Status`). The pump half was unread by the entire SPA until 2026-08-02.

Auth Denied

id `authDenied` · no route · stage index 0

STATUS

Under construction. Do not build UI for it. It has no manager charts at all, and since any denial escalates, those patients are worked directly on the board. This is a deliberate carve-out asserted by name in `insuranceCoverage.test.ts`.

05 Welcome Call & Subscription

Board 18410804557 · Welcome Call

two roles, two groups · Escalation `color_mm1x7997`, label `Escalation Required`

Welcome Call

id `welcomeCall` · `/welcome-call` · group_mm1wvq8p

ENTRY Welcome Call group.

DUE Not escalated **and** *Follow Up* `color_mm38w2tk` $\neq$ `Done`.

EXITS	OUTCOME	WRITES	DESTINATION
	Send to Monday	Call data columns, then Stage Advancer → <code>Review Profile</code>	<code>ADVANCES</code> Final Profile Confirmation

NOTES Out-of-pocket estimates come from `lib/welcomeCall/oopEstimator.ts`, which **mirrors Python on Railway by hand**. `PAYER_RATE_SCHEDULE` and the Medicaid / Medicare / NYSHIP / Humana special cases must be hand-synced with that backend — there is no automated drift check.

Final Profile Confirmation

id `finalConfirm` · `/final-confirm` · group_mm2x8jtj

ENTRY Final Confirm group.

DUE **Not escalated. That is the entire rule** — this group has **no follow-up filter at all**, unlike every other queue.

EXITS

OUTCOME

WRITES

DESTINATION

Confirm

Profile + order columns,
then Stage Advancer →
index 4 (Completed)

ADVANCES
automations move the
patient on

NOTES

The split-order duplicate path races a Monday "new item created" automation; the code re-writes flags defensively afterward (audit M6). Treat that path as known-fragile.

Board 18407459988 · Subscription

group `topics`

Subscription / Update Clinicals

ids `subscription` `updateClinicals`

ENTRY Every item in the group. **Both roles report the same number** — the raw group size.

DUE **No rule.** Nothing is filtered out.

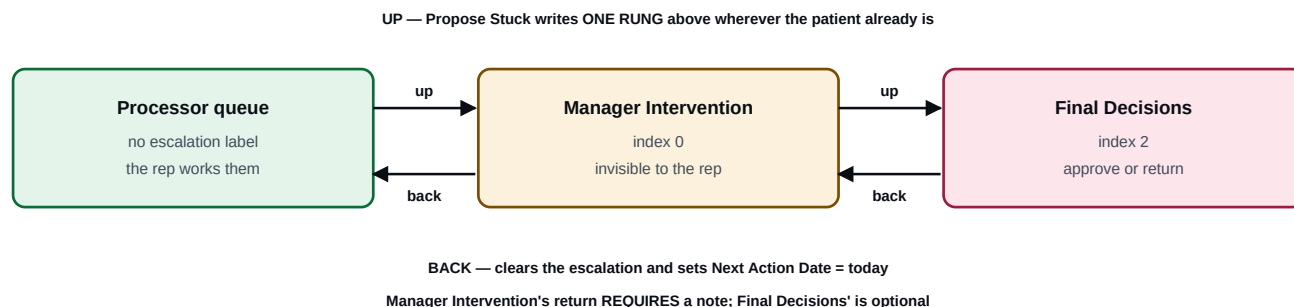
NOTES

KNOWN RISK — AUDIT H6

The Subscription send writes ~20 columns with retry but **no read-back verification**, and confetti fires even on partial failure. It does not use the four-phase protocol. Treat a green UI here as unverified.

06 The shared escalation ladder

Cross-cutting. Submit Auth, DVS and Doctor Appointments all use the same `stageActions.proposeStuckLevel`.



ACTION	WHO	EFFECT
Propose Stuck	Rep or manager	Writes one rung up from the patient's <i>current</i> label. An existing escalation, or a click from Manager Intervention, promotes to Final. Otherwise Submit Auth and DVS start at Manager; Benefits and Auth Outstanding go straight to Final. Reason stamped into the notes as <code>[Proposed Stuck ...]</code> .
Approve Stuck	Manager, Final Decisions	Main Stage Advancer → Stuck, then clear the escalation — in that order.
Send back to pipeline	Manager, either column	Optional stamped note (required in Manager Intervention), Next Action Date = today, escalation cleared. An escalated patient is invisible to the rep, so this is the only way back.
Send back to manager	Final Decisions, DVS rows only	Final → Manager index 0. The only Final → Manager transition in the system; everything else raises or clears.
Manager resolve	Manager working the patient	The three Insurance stage pages pass <code>managerResolve</code> when <code>?manager=1</code> . Completing the form clears the escalation and hands the patient back. Auto rules still run afterward, so a review that comes back denied re-escalates on the new facts.

THE INVARIANT THAT MUST NOT BREAK

Every escalated patient must land in some manager chart. Escalation removes them from the rep's queue and the role count, so a state matching no chart is invisible in the entire app. Manager chart bars key on board *facts* while escalations are *labels*, so the two drift — which is why each Insurance manager chart carries a population rule **wider** than its bars.

`lib/oversight/insuranceCoverage.test.ts` enumerates the reachable (stage × escalation) states and fails if one goes blind.

There is no Escalate toggle in the Insurance UI and one must not be reintroduced.

`components/samantha/EscalateButton.tsx` was deleted. A send writes the Escalation column only when an auto rule decides one; an auto rule only ever *raises*. The previous "manual toggle is the floor" rule read `p.escalated`, which is hydrated *from the board*, and re-wrote it on every send — silently promoting pending proposals past review, and silently clearing flags raised since the last poll.

07 Rules that live in more than one place

The counting contract. Change any of these and you change all of them in the same commit, or the queue and the count disagree with no error.

RULE	MUST AGREE ACROSS
Any queue / due / escalation rule	1. the role page (its <code>useMondayPatients</code> hook) 2. <code>src/hooks/useRoleCounts.ts</code> 3. <code>src/lib/oversight/oversightApi.ts</code> <code>CHART_FILTERS</code> 4. <code>scripts/snapshot-baseline.mjs</code> 5. <code>services/baseline-cron/index.mjs</code>
Chase Fax vs Email & Parachute split	All five above. This one has already regressed once — Email was moved back to chaseFax in the build-time baseline.
Profile Send Off split	All five, plus <code>CHART_ROUTES</code> in <code>OversightTab.tsx</code> .
Doctor Appointments sub-stage	All five. Without the branch in <code>useRoleCounts</code> the sub-stage falls through <code>if (!roleId) continue</code> and the patient is counted nowhere — no error, just invisible.
DVS chart label sets	The four board automations and the <code>dvs-manual-review</code> filter's <code>anyCols</code> . Disagreement escalates a patient into a chart that cannot list them.
A new Stedi result column	5 places inside the profile slice — <code>COL</code> and <code>READ_COLUMN_IDS</code> , the <code>Patient</code> type, <code>mondayItemToPatient</code> , <code>STEDI_SIGNATURE_KEYS</code> , and the render.
OOP estimator rates	<code>oopEstimator.ts</code> and the Railway financial backend. Hand-synced, no drift check.
HCPC modifiers	<code>submitAuthRules.ts</code> and the <code>claims-ui-tool</code> repo. Hand-synced.

Board-side rules the app depends on

- **Rename columns freely; never delete and recreate one.** Column IDs are the contract. A rename keeps the ID and reads keep working; a recreate assigns a new ID and reads return empty **silently**.
- **Status label text is load-bearing in specific places.** Sub-Stage matching, the Evaluate "Option A" dropdowns, and Clinicals Method all compare exact strings. Prefer index writes; where a label is compared, a casing change makes Monday create a duplicate label rather than fail.
- **A status write by index is silent if the index has no label.** Verify `settings_str` before adding an index — this is why Doctor Appointment is index 0 and not 12.
- **Never route a machine-parsed date through a text column across boards.** Monday's create-item automations type-sniff TEXT tokens: `2022-01-01` is re-rendered as `01 January 2022`. Dates ride in DATE columns, which copy verbatim.
- **Monday dates are timezone-naive ET.** Compare in ET, never with a bare `new Date()` in a UTC runtime — which is what Railway and CI are.

Read from the source on 11 August 2026, commit `b5a5f8f`. Where this contradicts `CLAUDE.md` — notably \$5.10 on the Profile Send Off split — this document reflects the code as of that commit.